

ABAC, governed tags and classification

Describing the data instead of configuring each table — and the newest material on this exam.

Stop configuring each table. **Describe the data** with tags, write one policy about the description, and every matching object is covered — including ones created next year.

Three capabilities that work together

01

Governed tags

An account-level vocabulary describing what data is.

02

Classification

Finds sensitive data and applies those tags automatically.

03

ABAC policies

Act on the tags, filtering rows and masking columns.

04

Separation of duties

Different people own the vocabulary, the tagging and the policies.

WHY THEY ARRIVED TOGETHER

Databricks released all three together, deliberately. Each is much less useful without the others.

Governed tags

- ◆ **Key-value pairs defined at the account level**, forming a controlled vocabulary.
- ◆ **Applied to catalogs, schemas, tables, columns, models and volumes**, and to workspace objects.
- ◆ **They describe characteristics:** sensitivity, classification, business domain. For example pii:ssn or sensitivity:confidential.
- ◆ **Permissions control who can apply which tags to which objects**, which is what makes them governed rather than free text.

ON THE EXAM

The word governed is the important one. Anyone can add a comment; only authorised people can apply a governed tag.

Tag inheritance, and its one exception

- ◆ Securables inherit tags from their parent catalog or schema by default.
- ◆ You can override inherited tags at every level except one.
- ◆ **Column tags do not inherit from the parent table.** They must be applied directly.
- ◆ Data creators automatically have **APPLY TAG** on objects they create, so they can add tags to their own data.

ON THE EXAM

That exception matters constantly. Tagging a table as containing PII does not tag its columns, and a column-matching policy will not fire.

A warning about tag content

- ◆ Tag data can be replicated globally.
- ◆ So do not use tag names or values that could compromise security, and never put personal or sensitive information in a tag.
- ◆ Tags describe the data. They are not the data.
- ◆ pii:ssn is a correct tag. A tag containing an actual identifier is not.

ON THE EXAM

Databricks flags this explicitly in its documentation. A tag is metadata, and metadata travels further than you might expect.

A policy attaches to a **catalog, schema or table**, and applies automatically to every object whose tags match its condition.

CREATE POLICY

CLAUSE	WHAT IT DOES
ON CATALOG SCHEMA TABLE	Where the policy attaches, and therefore its scope
ROW FILTER fn / COLUMN MASK fn	Which kind of policy, and the UDF that implements it
TO principal [EXCEPT principal]	Who it applies to, and who is exempted
FOR TABLES	Required
WHEN condition	Which securables match — usually a table-level tag test
MATCH COLUMNS condition AS alias	Which columns match, and a name to refer to them by
USING COLUMNS (args)	Arguments passed to the function, in order

ON THE EXAM

The policy name is scoped to the securable it is defined on. You need **MANAGE** on that securable, or ownership, to create it.

Only two functions in conditions

- ◆ `has_tag('key')` matches objects carrying that tag key.
- ◆ `has_tag_value('key', 'value')` matches objects carrying that key with that specific value.
- ◆ Those are the only user-facing functions supported in policy conditions.
- ◆ The camelCase forms `hasTag` and `hasTagValue` still work, for backward compatibility.

ON THE EXAM

Conditions are evaluated by the control plane on securable metadata, which is why the vocabulary is deliberately tiny.

WHEN and MATCH COLUMNS do different jobs

- ◆ **WHEN matches securables.** Which tables does this policy apply to at all?
- ◆ **MATCH COLUMNS matches columns.** Within those tables, which columns does it act on?
- ◆ **A column mask needs both,** plus ON COLUMN naming the alias to mask.
- ◆ **Confusing the two is the easiest mistake,** because both take a tag condition and they read alike.

ON THE EXAM

WHEN filters at table level, MATCH COLUMNS at column level. A policy can use either, or both together.

Two worked policies

POLICY	STATEMENT
Row filter	CREATE POLICY hide_eu_customers ON SCHEMA prod.customers ROW FILTER non_eu_region TO us_analysts FOR TABLES MATCH COLUMNS has_tag('geo_region') AS region USING COLUMNS (region)
Column mask	CREATE POLICY mask_ssn ON SCHEMA prod.customers COLUMN MASK mask_ssn TO us_analysts EXCEPT admins FOR TABLES MATCH COLUMNS has_tag_value('pii','ssn') AS ssn ON COLUMN ssn

ON THE EXAM

Read the row filter one carefully: it never names a table. It names a schema, and a tag. Every matching table is covered.

How a policy is evaluated

- ◆ Unity Catalog finds every policy whose scope covers the object being accessed.
- ◆ It checks whether the user is in the TO list and not in the EXCEPT list.
- ◆ It evaluates the WHEN condition against the object's tags, including inherited ones.
- ◆ If the policy applies, the filter or mask takes effect for that query.

ON THE EXAM

Every access, every time. Nothing is precomputed, so tagging a table this morning protects it this afternoon.

Requirements

- ◆ **MANAGE on the target securable, or ownership**, for every policy operation.
- ◆ **Databricks Runtime 16.4 or above, or serverless compute.**
- ◆ **Dedicated compute additionally needs fine-grained access control filtering enabled.**
- ◆ **Older runtimes cannot access tables secured by ABAC at all.**

ON THE EXAM

That last point is a hard failure, not a degradation. A cluster on an older runtime does not see unprotected data — it cannot read the table.

ABAC or table-level?

ABAC POLICIES

Describe once, apply everywhere

- Attach at catalog or schema level
- Cover new tables automatically as they are tagged
- Separation of duties between policy authors and stewards
- Queryable as policy metadata, so auditable at scale

TABLE-LEVEL FILTERS AND MASKS

Configure per object

- Attached with ALTER TABLE, managed by the table owner
- Apply to that one table or column only
- Right for genuinely per-table logic
- Simpler when there are only a few sensitive tables

THE TIEBREAKER

Databricks recommends ABAC when you need consistent rules across many tables. Table-level remains right for one-off logic.

When both apply

- ◆ **Same function on both sides:** execution is allowed.
- ◆ **Different functions:** access is blocked and an error is returned.
- ◆ **Even if the two functions would produce identical output.**
- ◆ **The design is fail-closed.** An ambiguous protection is refused rather than guessed at.

ON THE EXAM

So migrating table-by-table means removing the old policy as you add the new one, not leaving both in place.

Data classification

- ◆ **Automatically identifies sensitive data** such as personal and health information, and applies governed tags.
- ◆ **Built-in classifiers cover standards including GDPR and HIPAA.**
- ◆ **Custom classifiers learn from already-tagged columns**, extending detection to business-specific patterns.
- ◆ **New data is scanned automatically**, so sensitive data introduced later is caught.

ON THE EXAM

This is what makes the whole model work at scale. A policy is only as good as the tagging it acts on.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Consistent masking across hundreds of tables	An ABAC policy at catalog or schema level
New tables must be covered automatically	ABAC, as soon as they are tagged
A column-matching policy is not firing	Column tags do not inherit from the table
Only two functions work in a condition	has_tag and has_tag_value
Access blocked where both mechanisms apply	Different functions. Same function would be allowed.
An older cluster cannot read the table	ABAC needs DBR 16.4+, or serverless
Sensitive data must be found, not just protected	Data classification, applying governed tags

ON THE EXAM

Seven lines. The third is the one most likely to appear as a diagnosis question.

A bank must mask every column containing a national identifier across 400 tables in six catalogs, for everyone except the compliance group. New tables are created weekly by twelve different teams.

An engineer proposes writing a masking function and applying it to each column with ALTER TABLE.

What should they do instead?

- ☐ A The ALTER TABLE approach, scripted so it can be re-run weekly
- ☐ B Tag the columns with a governed tag and write one ABAC column mask policy per catalog
- ☐ C A dynamic view over each of the 400 tables
- ☐ D Revoke SELECT on all 400 tables from everyone except compliance

◆ PAUSE AND DECIDE

What should they do instead?

- A

 Scripted ALTER TABLE, re-run weekly
Works, and the protection is only ever as current as the last run. A table created on Tuesday is unmasked until the script runs.
- B

 Governed tags plus one ABAC policy per catalog
Correct. Six policies instead of hundreds of statements, and new tables are covered the moment they are tagged.
- C

 400 dynamic views
Recreates module 29's auditing problem at scale, and requires every consumer to be redirected to a view.
- D

 Revoke SELECT everywhere
Denies whole tables to people who need the other columns. Far broader than masking one column.

THE SIGNAL

'New tables are created weekly by twelve teams' is the deciding detail. Any approach that must be re-run has a window where data is exposed.

A team tags a table with `pii:true`, then writes a column mask policy on the schema using `MATCH COLUMNS has_tag_value('pii', 'true')`. The policy is created without error.

Querying the table returns unmasked values for everyone. `SHOW POLICIES` confirms the policy exists on the schema.

Why is nothing being masked?

- ☐ A The policy needs to be attached to the table rather than the schema
- ☐ B The tag was applied to the table, and column tags do not inherit from the parent table; the columns must be tagged directly
- ☐ C `has_tag_value` is not supported in `MATCH COLUMNS`
- ☐ D The policy requires a `WHEN` clause to function

◆ PAUSE AND DECIDE

Why is nothing being masked?

- A** Attach it to the table
Schema-level attachment is normal and correct. The scope is fine; the tags are in the wrong place.
- C** has_tag_value is unsupported there
Both has_tag and has_tag_value are supported in conditions. They are the only two, and this is one of them.

- B** Column tags do not inherit from the table
Correct. Inheritance can be overridden at every level except columns — column tags must be applied directly, so MATCH COLUMNS finds nothing.
- D** A WHEN clause is required
WHEN is optional. It narrows which securables match; its absence means the policy considers all of them.

THE SIGNAL

The policy is correct and matches nothing. MATCH COLUMNS looks at column tags, and the only tag applied was on the table.

A team is migrating from table-level column masks to ABAC. On one table they add an ABAC column mask policy while the existing ALTER TABLE mask is still in place. Both use different functions, though both redact to the same fixed string.

Queries against that table now fail with an error rather than returning masked data.

What is happening?

- A When both mechanisms apply different functions, access is blocked — even if the output would be identical
- B ABAC policies cannot be applied to tables that already have masks; the ABAC policy was never created
- C The two masks are being applied in sequence, double-masking the value
- D The table needs OPTIMIZE before the new policy takes effect

◆ PAUSE AND DECIDE

What is happening?

- A

 Different functions block access
Correct. Same function on both sides is allowed; different functions are blocked with an error, regardless of whether the output matches.
- B

 The policy was never created
Both can coexist — the same function on both sides is explicitly allowed. Creation is not what failed.
- C

 Double-masking
The system does not chain them. It refuses rather than composing two protections it cannot reconcile.
- D

 Run OPTIMIZE first
OPTIMIZE concerns file layout. Policies are evaluated at query time from metadata and need no maintenance operation.

THE SIGNAL

'Though both redact to the same fixed string' is placed to tempt you. The system does not compare outputs — only the functions.

A healthcare company must find and protect health information across an estate nobody has fully catalogued. They have a compliance team who understand HIPAA, a data platform team who write policies, and hundreds of data producers.

A manager proposes the compliance team manually review every column and hand a list to the platform team.

What is the better design?

- | | |
|--|---|
| <p>A The manual review, since compliance expertise cannot be automated</p> | <p>B Automated classification applies governed tags; compliance owns the tag vocabulary; the platform team writes ABAC policies against those tags</p> |
| <p>C Give the compliance team ownership of every table so they can apply masks directly</p> | <p>D Have each data producer write their own column masks</p> |

◆ PAUSE AND DECIDE

What is the better design?

- A

 Manual review by compliance
Does not scale to an uncatalogued estate, and is a snapshot — new columns appear continuously and the review is immediately stale.
- B

 Classification tags, compliance owns the vocabulary, platform writes policies
Correct. This is exactly the separation of duties the three capabilities were designed to enable, with built-in HIPAA classifiers doing the discovery.
- C

 Compliance owns every table
Makes a compliance team responsible for operating data they did not create. It also concentrates rather than separates duties.
- D

 Each producer writes their own masks
Hundreds of independent interpretations of HIPAA, with no consistency and nothing centrally auditable.

THE SIGNAL

Three roles, three capabilities. Compliance defines what sensitive means, classification finds it, and policies act on it.

Tag the data, write one policy about the tag. **Coverage then follows the description, not the configuration** — including for tables that do not exist yet.